`markdown2html5-base`

Full Markdown Functionality Reference

This reference covers all 20 feature categories:

- headings (with custom IDs),
- inline styling (bold/italic/strikethrough/highlight/sub/sup/underline/code),
- links and images,
- fenced code blocks,
- horizontal rules,
- lists (unordered/ordered/task),
- blockquotes,
- tables (with alignment and table foot),
- definition lists,
- footnotes,
- language markers,
- ruby annotations,
- emoji shortcodes,
- typography/legal marks,
- hard line breaks,
- HTML comments,
- YAML front matter,
- predefined CSS styles,
- backslash escaping,
- and paragraph handling.

Programmatic Usage

```
from markdown2html5_base import MarkdownToHTML
converter = MarkdownToHTML()
html = converter.convert("# Hello")
print(html)
```

1. Headings (H1-H6)

Markdown	Output HTML
`# Heading 1`	`<h1>Heading 1</h1>`
`## Heading 2`	`<h2>Heading 2</h2>`
`### Heading 3`	`<h3>Heading 3</h3>`
`#### Heading 4`	`<h4>Heading 4</h4>`
`##### Heading 5`	`<h5>Heading 5</h5>`
`###### Heading 6`	`<h6>Heading 6</h6>`

Custom ID: `## Section {#sec1}` => `<h2 id="sec1">Section</h2>`

2. Inline Text Styling

Markdown	Output HTML
`***bold italic***`	`<strong><em>bold italic</em></strong>`
`___bold italic___`	`<strong><em>bold italic</em></strong>`
`**bold**`	`<strong>bold</strong>`
`__bold__`	`<strong>bold</strong>`
`*italic*`	`<em>italic</em>`
`_italic_`	`<em>italic</em>`
`~~strikethrough~~`	`<s>strikethrough</s>`
`==highlight==`	`<mark>highlight</mark>`
`~subscript~`	`<sub>subscript</sub>`
`^^underline^^`	`<u>underline</u>`
`^superscript^`	`<sup>superscript</sup>`
`` `code` ``	`<code>code</code>`

3. Links and Images

Markdown	Output HTML
`[text](url)`	`<a href="url">text</a>`
`![alt](src)`	`<img src="src" alt="alt">`
`![alt](src "Title")`	`<img src="/home/nobus/OpenCode/markdown2html5-base/markdown2html5-base/src" alt="alt" title="Title">`

4. Fenced Code Blocks

A language tag after the opening fence (three backticks before and after the code) is rendered as a `<div class="code-lang">&sol;python&sol;</div>` label above the `<code>` element:

```
<div class="code-lang">&sol;python&sol;</div><pre><code>def hello():
    print("Hi")</code></pre>
```

Without a language tag, the code renders plainly as `<pre><code>`.

5. Horizontal Rules

`---` / `***` / `___` (3+ chars) => `<hr>`

6. Lists

1. **Unordered:** `* Item` or `- Item` => `<ul><li>Item</li></ul>`
2. **Ordered:** `1. Item` => `<ol><li>Item</li></ol>`
3. **Task:**
   `* [ ] todo` => `<li><input type="checkbox" disabled> todo</li>`
   `* [x] done` => `<li><input type="checkbox" checked disabled> done</li>`

7. Blockquotes

`> text` => `<blockquote><p>text</p></blockquote>`: blank lines within a blockquote split into separate `<p>` tags.

8. Tables

Supports `<thead>`, `<tbody>`, `<tfoot>`, and alignment:

Separator	Alignment
`:---`	left
`:---:`	center
`---:`	right

Footer: a row of `=` signs in the separator columns after body rows renders `<tfoot>`.

```
| Product  | Qty | Price |
| :------- | :-: | ----: |
| Apples   | 2   | $3.00 |
| Bananas  | 3   | $1.50 |
| Cherries | 1   | $4.00 |
|==========|=====|=======|
| Total    | 6   | $8.50 |
```

=>

```
<table>
  <thead>
    <tr>
      <th style="text-align:left;">Product</th>
      <th style="text-align:center;">Qty</th>
      <th style="text-align:right;">Price</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td style="text-align:left;">Apples</td>
      <td style="text-align:center;">2</td>
      <td style="text-align:right;">$3.00</td>
    </tr>
    <tr>
      <td style="text-align:left;">Bananas</td>
      <td style="text-align:center;">3</td>
      <td style="text-align:right;">$1.50</td>
    </tr>
    <tr>
      <td style="text-align:left;">Cherries</td>
      <td style="text-align:center;">1</td>
      <td style="text-align:right;">$4.00</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td style="text-align:left;">Total</td>
      <td style="text-align:center;">6</td>
      <td style="text-align:right;">$8.50</td>
    </tr>
  </tfoot>
</table>
```

9. Definition Lists

```
Term
: Definition 1
: Definition 2
```

=>

```
<dl>
  <dt>Term</dt>
  <dd>Definition 1</dd>
  <dd>Definition 2</dd>
</dl>
```

10. Footnotes

Reference: `[^1]` => `<sup id="fnref:1"><a href="#fn:1" class="footnote-ref">1</a></sup>`

Definition: `[^1]: Text` at bottom => rendered in `<div class="footnotes"><ol>...</ol></div>`

11. Language Markers

Annotate blocks or inline text with a language using a valid BCP 47 tag (e.g. `de`, `fr`, `zh-Hans`).

- **Block level:** Place `{:lang}` (with a space after it) at the very start of a line, before the Markdown tag. It renders as the global `lang` attribute on the resulting element and needs no closing marker:

```
{:de} # Überschrift
{:fr} Un paragraphe français.
{:ru} - Пункт списка
{:de} > Ein Zitat
{:de} | Kopf  | Kopf  |
      | ----- | ----- |
      | Zelle | Zelle |
```

=>

```
<h1 lang="de">Überschrift</h1>
<p lang="fr">Un paragraphe français.</p>
<ul lang="ru">
  <li>Пункт списка</li>
</ul>
<blockquote lang="de">...</blockquote>
<table lang="de">...</table>
```

- **Inline level:** Wrap text with `{:lang}...{:}` to render a `<span lang="...">`:

```
A French phrase {:fr}"L'État c'est moi"{:} is traditionally attributed to King Louis XIV of France.
```

=>

```
<p>A French phrase <span lang="fr">“L‘État c’est moi”</span> is traditionally attributed to King Louis XIV of France</p>
```

12. Ruby Annotations

`{日本語|にほんご}` => `<ruby>日本語<rp>(</rp><rt>にほんご</rt><rp>)</rp></ruby>`

13. Emoji Shortcodes

- `:joy:` 😂
- `:smile:` 😄
- `:heart:` ❤️
- `:thumbsup:` 👍
- `:thumbsdown:` 👎
- `:wink:` 😉
- `:tada:` 🎉
- `:rocket:` 🚀
- `:fire:` 🔥
- `:star:` ⭐
- `:cry:` 😢
- `:thinking:` 🤔
- `:100:` 💯
- `:sparkles:` ✨
- `:eyes:` 👀
- `:bulb:` 💡
- `:warning:` ⚠️
- `:ok:` 👌
- `:check_mark:` ✔

14. Typography and Legal Marks

Name	Input	HTML Output
Copyright	`(c)`	`&copy;`
Trademark	`(tm)`	`&trade;`
Registered Trademark	`(r)`	`&reg;`
Plus-Minus	`+/-`	`&plusmn;`
Not Equal To	`!=`	`&ne;`
Logical Equivalence	`<=>`	`&hArr;`
Less-Than or Equal To	`<=`	`&le;`
Greater-Than or Equal To	`>=`	`&ge;`
Left Arrow	`->`	`&rarr;`
Right Arrow	`<-`	`&larr;`
Up Arrow	`:uparrow:`	`&uarr;`
Down Arrow	`:dnarrow:`	`&darr;`
Logical Implication	`=>`	`&rArr;`
One-Half	`1/2`	`&frac12;`
One-Third	`1/3`	`&frac13;`
Two-Thirds	`2/3`	`&frac23;`
One-Quarter	`1/4`	`&frac14;`
Three-Quarters	`3/4`	`&frac34;`
Solidus (Slash)	`:slash:`	`&sol;`
Reverse Solidus (Backslash)	`:bslash:`	`&bsol;`
Left Angle Quote	`<<`	`&laquo;`
Right Angle Quote	`>>`	`&raquo;`
Smart Double Quotes	`"text"`	`&ldquo;text&rdquo;`
Smart Single Quotes	`'text'`	`&lsquo;text&rsquo;`
Straight Apostrophe	`'`	`&apos;`
Em Dash	`---`	`&mdash;`
En Dash	`--`	`&ndash;`
Ellipsis	`...`	`&hellip;`

15. Hard Line Breaks

End line with two spaces or backslash: `<br />`

16. HTML Comments

`[comment]: #` => `<!--comment-->`

17. YAML Front Matter

If the file begins with a YAML front matter block between `---` lines, the converter emits a complete HTML5 document with the metadata in `<head>` and the body content between `<body>` tags. Without front matter, the output stays a bare HTML fragment. The `--css` option (or `include_css=True`) embeds the default `<style>` block regardless of whether front matter is present.

```yaml
---
lang: en
title: My Document
author: Jane Doe
description: A short description.
keywords: python, markdown, html5
published: 2026-08-09
---
```

- `lang` becomes the `<html lang="...">` attribute (a valid BCP 47 tag).
- `title` becomes the `<title>` element.
- `author`, `description`, `keywords`, and `published` become `<meta name="..." content="..." />` tags.
- A default `<style>` block with viewing-friendly CSS is embedded in `<head>`.

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="author" content="Jane Doe" />
    <meta name="description" content="A short description." />
    <meta name="keywords" content="python, markdown, html5" />
    <title>My Document</title>
    <meta name="published" content="2026-08-09" />
    <style>
      /* default viewing-friendly CSS */
    </style>
  </head>
  <body>
    ...
  </body>
</html>
```

Any other keys are ignored, and if the file has no front matter at all, the output remains a bare fragment (unless `--css` / `include_css=True` is used, in which case it becomes a full document).

18. Backslash Escaping

Escape any special char: `\*`, `\#`, `\[`, etc.

Escapable: \ ` * _ { } [ ] ( ) # + - . ! | ~ ^ = : < >

19. Paragraphs

Consecutive text lines merge into `<p>`. Blank lines separate paragraphs.

Empty line after list close => `<!-- -->` preserves whitespace.

20. CSS Styles

The converter (with `--css` option or `include_css=True`) embeds a default `<style>` block in `<head>` that provides viewing-friendly styling, regardless of YAML front matter. This predefined CSS rulles includes:

```css
body {
  padding: 20px;
  font-family:
    "Noto Serif",
    "Liberation Serif",
    "Times New Roman",
    Times,
    serif;
  font-size: 18px;
  line-height: 1.4;
  color: #000000;
}
h1, h2, h3, h4, h5, h6 {
  margin-top: 1.2em;
  margin-bottom: 0.6em;
  font-family:
    "Noto Sans",
    "Liberation Sans",
    Arial,
    sans-serif;
  font-weight: bold;
}
h1 { font-size: 32px; }
h2 { font-size: 28px; }
h3 { font-size: 24px; }
h4 { font-size: 20px; }
h5 { font-size: 18px; }
h6 {
  font-size: 18px;
  font-style: italic;
}
hr {
  height: 4px;
  margin: 20px 0;
  border: none;
  background-color: #000000;
}
li {
  position: relative;
  padding-left: 20px;
}
dt { font-weight: bold; }
dd {
  position: relative;
  margin-left: 0;
  padding-left: 20px;
  font-style: italic;
}
blockquote {
  margin-left: 0;
  padding-left: 20px;
  border-left: 8px solid #f5f5f5;
}
mark {
  padding: 0 2px;
  border-radius: 4px;
  background-color: #ffff00;
}
a:link { color: #0000cd; }
a:visited { color: #9400d3; }
```

```css
a:hover, a:focus {
  outline: none;
  color: #000080;
}
a:active { color: #dc143c; }
code {
  padding: 2px 4px;
  border-radius: 4px;
  font-family:
    "Noto Sans Mono",
    "Liberation Mono",
    "Courier New",
    Courier,
    monospace;
  font-size: 0.9em;
  line-height: 1;
  background-color: #f5f5f5;
}
pre {
  max-width: 100%;
  margin: 0;
  padding: 20px;
  border: 1px solid #000000;
  background-color: #f5f5f5;
  overflow: auto;
  scrollbar-color: #000000 transparent;
}
pre > code {
  display: block;
  margin: 0;
  padding: 0;
  border: none;
  border-radius: 0;
  line-height: 1.2;
  background-color: transparent;
  overflow: visible;
}
div.code-lang {
  display: block;
  padding: 10px 20px;
  font-family:
    "Noto Sans Mono",
    "Liberation Mono",
    "Courier New",
    Courier,
    monospace;
  font-size: 0.9em;
  line-height: 1;
  background-color: #000000;
  color: #ffffff;
  font-weight: bold;
}
table {
  margin: 20px 0;
  border-collapse: collapse;
}
th, td {
  padding: 10px 12px;
  border: 1px solid #000000;
}
th { font-weight: bold; }
```

```
thead tr {
  background-color: #000000;
  color: #ffffff;
}
tfoot tr {
  background-color: #f5f5f5;
  font-style: italic;
}
ruby { ruby-position: over; }
rt {
  letter-spacing: 0.05em;
  font-size: 0.55em;
  line-break: strict;
}
rp { display: none; }
span[lang="ja"] {
  font-family:
    "Noto Serif CJK JP",
    "Source Han Serif JP",
    "源ノ明朝",
    "Source Han Serif",
    "Hiragino Mincho ProN",
    "Hiragino Mincho Pro",
    "IPAexMincho",
    "IPAMincho",
    "MS PMincho",
    "MS Mincho",
    serif;
}
span[lang="zh-CN"], span[lang="zh-Hans"] {
  font-family:
    "Noto Serif CJK SC",
    "Source Han Serif SC",
    "思源宋体",
    "Source Han Serif CN",
    "Source Han Serif",
    "Songti SC",
    "FandolSong",
    "WenQuanYi Bitmap Song",
    "SimSun",
    serif;
}
span[lang="zh-TW"], span[lang="zh-Hant"] {
  font-family:
    "Noto Serif CJK TC",
    "Source Han Serif TC",
    "思源宋體",
    "Source Han Serif TW",
    "Source Han Serif",
    "Apple LiSung",
    "LiSong Pro",
    "HanaMinA",
    "PMingLiU",
    "MingLiU",
    serif;
}
span[lang="zh-HK"] {
  font-family:
    "Noto Serif CJK HK",
    "Source Han Serif HK",
    "思源宋體 香港",
```

```
    "思源宋體",
    "Source Han Serif",
    "Apple LiSung",
    "LiSong Pro",
    "HanaMinA",
    "MingLiU_HKSCS",
    "PMingLiU",
    "MingLiU",
    serif;
}
span[lang="ko"] {
  font-family:
    "Noto Serif CJK KR",
    "Source Han Serif KR",
    "본명조",
    "Source Han Serif",
    "AppleMyungjo",
    "UnBatang",
    "은바탕",
    "Batang",
    serif;
}
```